

Large-scale Messaging System: 업무 플랫폼으로 확장된 메신저 재설계

메시징 시스템은 socket 연결이 아니라 room state, delivery, sync, 장애 추적을 다루는 운영 시스템이다.

요약

사내 메신저가 고객사 업무 플랫폼으로 확장되면서 단순 채팅 서버 구조를 넘어 gateway, broker, DB write, cache, room sync, sequence, 장애 추적을 다시 설계해야 했다. 핵심은 메시지를 빠르게 보내는 것뿐 아니라, 재접속 이후 상태를 정확히 맞추고 장애 상황을 설명할 수 있는 구조를 만드는 것이었다.

이 프로젝트에서 정리한 기준은 명확하다. 실시간 시스템은 평균 처리량보다 tail latency, 재접속 동기화, 장애 복구 경로, 상태 일관성이 중요하다.

문제 정의

메신저가 내부 도구일 때와 고객사 업무 플랫폼일 때 요구 수준은 다르다. 고객사, 부서, 권한, 대화방, 첨부, 읽음 상태, 알림, 검색, 보관 정책이 결합되면 단순 WebSocket broadcast로 해결되지 않는다.

주요 문제는 다음과 같았다.

- socket 단절 후 사용자의 상태를 정확히 복구해야 한다.
- 신규 메시지뿐 아니라 수정·삭제·읽음 상태도 동기화해야 한다.
- 방별 message sequence와 변경 이력이 필요하다.
- DB write 병목과 cache invalidation을 설계해야 한다.
- gateway, broker, DB, cache 장애를 구분해 추적해야 한다.
- 고객사별 traffic과 권한 정책을 분리해야 한다.

책임 경계

계층	책임
Gateway	socket 연결, session lifecycle, 인증, ingress event를 처리한다.
Broker	메시지 fan-out, ordering, backpressure, 재시도 흐름을 담당한다.

계층	책임
DBRuntime	메시지 저장, lazy batch, shard, transaction 경계를 관리한다.
CacheRuntime	room state, unread, presence, hot data cache를 관리한다.
Sync API	재접속 이후 신규·수정·삭제 상태를 정합성 있게 내려준다.
Monitoring	gateway, broker, DB, cache 지표를 연결한다.

핵심 설계 결정

Gateway는 연결 서버가 아니라 상태 경계로 보았다

Gateway는 WebSocket을 열어두는 서버가 아니다. 사용자가 언제 연결되고, 어떤 ticket으로 인증되었고, 어느 room에 참여했고, 어떤 gateway에 붙어 있는지 관리하는 상태 경계다. 이 정보가 불명확하면 장애 상황에서 “메시지를 보냈는데 왜 못 받았는가”를 설명할 수 없다.

Kafka와 broker 구조를 검토했다

대규모 fan-out과 비동기 처리에서는 broker 선택이 중요하다. Kafka는 durability, consumer group, partition 기반 처리에 강점이 있다. Pulsar는 topic scale과 multi-tenancy 측면에서 장점이 있다. 이 프로젝트에서는 운영 중인 시스템과 연동, 장애 추적, 기존 경험, 처리 모델을 기준으로 Kafka 중심 구조를 우선 검토했다.

핵심은 broker 이름이 아니라 메시지 흐름을 어떻게 분리하는가였다.

```
socket ingress
-> gateway event
-> broker topic
-> persist worker
-> fan-out worker
-> sync state update
-> audit / monitoring
```

Room Change Log를 두었다

신규 메시지는 비교적 단순하다. 더 어려운 것은 중간에 수정·삭제된 메시지와 읽음 상태, 첨부 상태, 알림 상태를 재접속 사용자에게 정확히 맞추는 것이다. 그래서 메시지 전문과 별개로 room change log를 고려했다.

room에 입장하면 전체 데이터를 무조건 내려주는 것이 아니라, 마지막 sync point 이후의 변경 이력을 new , updated , deleted 같은 상태로 내려주는 방식이 적합했다. 이 구조는 모바일 재접속, offline 상태, 다중 device sync에서 중요하다.

sequence는 전역보다 방 단위가 중요했다

전역 sequence 서버는 명확해 보이지만 운영 비용이 크고 병목이 될 수 있다. 메시징에서는 대부분의 ordering 요구가 room 단위로 발생한다. 따라서 방 단위 sequence와 shard 기준을 우선 검토했다.

전역 순서가 필요한 이벤트와 room 내부 순서가 필요한 이벤트를 구분하지 않으면 구조가 과도하게 복잡해진다.

버린 선택과 이유

메신저를 단순 WebSocket broadcast로 구현하는 방식은 업무 플랫폼에는 충분하지 않았다. 연결된 사용자에게만 메시지를 보내는 것은 쉬우나, 재접속 이후 누락된 메시지, 읽음 상태, 방 변경 이력, 알림 재처리, multi-device sync를 설명하기 어렵다.

단일 DB transaction으로 모든 메시징 상태를 처리하는 방식도 한계가 있었다. message write, fanout, notification, unread count, search index, audit은 처리 목적과 지연 허용 범위가 다르다. 모든 처리를 동기 transaction에 묶으면 tail latency와 장애 영향 범위가 커진다.

전역 sequence를 기준으로 전체 메시지를 정렬하는 방식도 우선하지 않았다. 운영에서 중요한 것은 전체 시스템의 절대 순서보다 방 단위의 순서 보장, gap 감지, 재동기화 가능성이다.

재접속을 기준으로 본 운영 품질

메신저 품질은 평상시 전송 성공률만으로 판단할 수 없다. 업무용 메신저에서는 network 변경, 브라우저 sleep, 모바일 전환, gateway 재시작 이후에도 상태를 맞춰야 한다.

확인 항목	목적
마지막 수신 sequence	client가 어디까지 받았는지 판단한다.
room change log	방 초대, 퇴장, 권한 변경 이후 sync 범위를 계산한다.
unread 재계산 기준	cache 값과 DB 값이 어긋날 때 복구한다.
gateway session 상태	연결 문제와 application 처리 문제를 분리한다.
message idempotency	재시도와 중복 전송을 안전하게 처리한다.

DBRuntime과 CacheRuntime

메시징 시스템에서는 DB 저장과 cache 갱신이 병목으로 전환되는 구간이 자주 발생한다. 모든 요청을 즉시 DB에 반영하면 tail latency가 늘고, 장애 시 write 폭주가 발생한다. 반대로 비동기화만 강조하면 사용자에게 보이는 상태와 저장 상태가 어긋난다.

DBRuntime은 lazy insert, batch flush, shard, retry, DLQ를 관리하는 실행 계층으로 보았다.

CacheRuntime은 room state, unread count, presence, hot room data를 관리하되, invalidate와 재구성 경로를 함께 가져야 했다.

시스템 설계자의 그림

메시징 시스템의 설계 기준은 전송 성공이 아니라 재접속 이후의 복구 가능성이다. gateway, broker, DB, cache를 각각 따로 최적화하면 빠르게 보일 수 있지만, 사용자가 마지막으로 받은 sequence 이후 무엇을 받아야 하는지 설명하지 못하면 업무 플랫폼으로는 부족하다.

운영 검증 {#운영-검증 }

검증 기준은 평균 TPS 하나로 끝나지 않았다. 다음 기준을 함께 봐야 했다.

- P50, P90, P99 latency가 분리되어 관측되는가.
- socket 단절 후 sync API가 정확한 변경분을 내려주는가.
- gateway 장애 시 session과 presence가 정리되는가.
- broker 지연과 DB write 지연을 구분할 수 있는가.
- cache miss와 stale state를 복구할 수 있는가.
- 특정 room hot spot이 전체 시스템을 흔들지 않는가.

역할과 책임

Messaging System의 gateway/broker/DB/cache 책임 경계, room change log, sequence 기준, sync API, lazy DB write, cache invalidation, 장애 추적 지표의 설계 방향을 정리하고 핵심 구조를 검토했다.

설계 결론

메시징 시스템은 실시간 전송 기능이 아니라 상태 동기화 시스템이다. 메시지를 보낸 순간보다 사용자가 다시 들어왔을 때 무엇을 받아야 하는지가 더 중요하다. 따라서 gateway, broker, DB, cache, sync API를 하나의 운영 흐름으로 설계해야 한다.